

Chapter 33 — Sweep Line

0. What You Will Learn in This Chapter

Sweep Line is an advanced interval/timeline pattern.

It is used when many events happen over a line, usually time or coordinate, and you need to know what is active at each point.

Sweep Line problems often ask:

- maximum overlap
- number of active intervals
- whether capacity is exceeded
- merged painted segments
- calendar booking conflicts
- skyline outline
- total covered length
- range additions over time
- intersections of rectangles
- meeting room count
- population over years

By the end of this chapter, you should understand:

- What sweep line means
- Why events are useful
- How intervals become start/end events
- How to sort events
- How to handle ties correctly
- How sweep line relates to difference array
- How sweep line relates to intervals
- How sweep line relates to ordered map
- How to count active intervals
- How to detect maximum overlap
- How to process sparse coordinates
- How to solve calendar problems
- How to solve skyline-style problems
- How to avoid boundary mistakes

Core problems covered in this chapter:

1. Meeting Rooms II, sweep line version
2. Car Pooling
3. Maximum Population Year
4. My Calendar I
5. My Calendar II
6. My Calendar III
7. Divide Intervals Into Minimum Number of Groups
8. Number of Flowers in Full Bloom
9. Describe the Painting
10. Amount of New Area Painted Each Day
11. The Skyline Problem
12. Rectangle Area II
13. Employee Free Time, sweep/merge view
14. Range Addition, difference-array connection
15. Corporate Flight Bookings, difference-array connection

1. What Is Sweep Line?

Sweep Line is a technique where we imagine a vertical line moving from left to right across coordinates or time.

As the line moves, events happen.

Example intervals:

[1, 4]

[2, 5]

[7, 9]

Events:

time 1: interval starts

time 2: interval starts

time 4: interval ends

time 5: interval ends

time 7: interval starts

time 9: interval ends

As we sweep from left to right, we maintain:

active intervals

At each event, active count changes.

2. Why Sweep Line Exists

Many interval problems are hard if you compare every pair of intervals.

Example:

Find maximum number of overlapping intervals.

Brute force:

for every interval:

compare with every other interval

This is $O(n^2)$.

Sweep line transforms intervals into events and processes them in sorted order.

For each interval:

[start, end)

create:

(start, +1)

(end, -1)

Then sort events and scan.

This gives:

$O(n \log n)$

3. Core Sweep Line Idea

For every interval:

[start, end]

or:

[start, end)

convert it into two events:

start event

end event

Then process events in order.

Maintain a running variable:

active

When interval starts:

active++

When interval ends:

active--

The maximum value of active tells maximum overlap.

4. Sweep Line vs Difference Array

Sweep Line and Difference Array are closely related.

Difference Array

Use when coordinates are small and integer-indexed.

Example:

years 1950 to 2050

locations 0 to 1000

Use array:

```
vector<int> diff(size);
```

Sweep Line

Use when coordinates are large or sparse.

Example:

time can go up to 1e9

Use map or sorted events:

```
map<int, int> events;
```

or:

```
vector<pair<int, int>> events;
```

Comparison

Small coordinate range	Difference Array
Large sparse coordinates	Sweep Line
Need sorted event processing	Sweep Line
Need final value at every index	Difference Array
Need max overlap only	Sweep Line
Need dynamic booking	Ordered map sweep

5. Sweep Line vs Interval Problems

Interval problems usually process whole intervals.

Sweep line processes interval boundaries.

Example:

Interval problem:

merge overlapping intervals

Sweep line problem:

how many intervals are active at every point?

Use interval merging when you need union of intervals.

Use sweep line when you need active counts or event-based changes.

6. When to Use Sweep Line

Use sweep line when the problem asks for:

6.1 Maximum Overlap

Examples:

- Meeting Rooms II
- Minimum Number of Groups
- My Calendar III

6.2 Capacity Over Time

Examples:

- Car Pooling
- Corporate Flight Bookings
- Range Addition

6.3 Calendar Booking

Examples:

- My Calendar I
- My Calendar II
- My Calendar III

6.4 Population / Active Count

Examples:

- Maximum Population Year
- Number of Flowers in Full Bloom

6.5 Painted / Covered Segments

Examples:

- Describe the Painting
- Amount of New Area Painted Each Day
- Rectangle Area II

6.6 Geometry

Examples:

- The Skyline Problem
 - Rectangle Area II
 - line segment union
-

7. When NOT to Use Sweep Line

Do not use sweep line when:

7.1 You Only Need Simple Merge

Use interval merge.

Example:

Merge Intervals

7.2 You Need Shortest Path

Use BFS or Dijkstra.

7.3 You Need Dynamic Connectivity

Use Union Find.

7.4 You Need Range Queries With Updates

Use Fenwick Tree or Segment Tree.

7.5 You Need All Pairwise Intersections

Sweep line may help, but simple sweep may not be enough. More advanced geometry may be needed.

8. Mental Models

8.1 Timeline Events

Every interval starts and ends.

The sweep line walks through time and updates active state.

8.2 Active Count

At any coordinate:

active = number of intervals currently open

8.3 Start Adds, End Removes

For interval [start, end):

events[start] += 1
events[end] -= 1

8.4 Sparse Difference Array

A map-based sweep line is like a difference array where only changed positions are stored.

```
map<int, int> events;  
events[start]++;  
events[end]--;
```

9. Pattern Recognition Checklist

Ask:

16. Do intervals start and end over time?
17. Do I need maximum number of overlaps?
18. Do I need active count at each point?
19. Do I need capacity check over time?
20. Are coordinates large or sparse?
21. Can every range be represented by start/end events?
22. Are there many bookings?
23. Do start/end ties matter?
24. Is the problem about timeline simulation?
25. Would pairwise interval comparison be too slow?

If yes, sweep line may apply.

10. C++ Tools Needed

10.1 Vector of Events

```
vector<pair<int, int>> events;
```

For each interval:

```
events.push_back({start, +1});  
events.push_back({end, -1});
```

Sort:

```
sort(events.begin(), events.end());
```

10.2 Ordered Map of Events

```
map<int, int> events;
```

```
events[start]++;  
events[end]--;
```

Then scan:

```
int active = 0;  
  
for (auto& [time, delta] : events) {
```

```
    active += delta;
}
```

10.3 Min Heap for Active Ends

Sometimes heap is easier than explicit event map.

```
priority_queue<int, vector<int>, greater<int>>> minHeap;
```

Used in:

- Meeting Rooms II
- Divide Intervals Into Groups

10.4 Multiset for Dynamic Heights

Used in Skyline:

```
multiset<int> heights;
```

11. Core Templates

Template 1 — Basic Sweep Line With Events

```
int maxOverlap(vector<vector<int>>& intervals) {
    vector<pair<int, int>> events;

    for (auto& interval : intervals) {
        events.push_back({interval[0], +1});
        events.push_back({interval[1], -1});
    }

    sort(events.begin(), events.end());

    int active = 0;
    int best = 0;

    for (auto& [time, delta] : events) {
        active += delta;
        best = max(best, active);
    }

    return best;
}
```

This assumes half-open intervals [start, end) where end should be processed before start at same time.

Because -1 sorts before +1, pair<int,int> sorting handles that naturally.

Template 2 — Sweep Line With Map

```

int maxOverlap(vector<vector<int>>& intervals) {
    map<int, int> events;

    for (auto& interval : intervals) {
        events[interval[0]]++;
        events[interval[1]]--;
    }

    int active = 0;
    int best = 0;

    for (auto& [time, delta] : events) {
        active += delta;
        best = max(best, active);
    }

    return best;
}

```

Use this when coordinates are sparse or updates are dynamic.

Template 3 — Closed Interval Sweep

For closed intervals [start, end], if using integer coordinates and you want intervals ending at end to remain active at end, then one common difference-array style is:

```

events[start]++;
events[end + 1]--;

```

But this only works safely when coordinates are integer and end + 1 is meaningful. For general coordinate sweep, handle ties manually.

Template 4 — Active Interval Heap

```

sort(intervals.begin(), intervals.end());

priority_queue<int, vector<int>, greater<int>> minHeap;

for (auto& interval : intervals) {
    int start = interval[0];
    int end = interval[1];

    while (!minHeap.empty() && minHeap.top() <= start) {
        minHeap.pop();
    }
}

```

```
    minHeap.push(end);  
}
```

Used when active intervals expire by end time.

Template 5 — Dynamic Calendar Sweep

```
map<int, int> events;  
  
bool book(int start, int end) {  
    events[start]++;  
    events[end]--;  
  
    int active = 0;  
  
    for (auto& [time, delta] : events) {  
        active += delta;  
  
        if (active >= 3) {  
            events[start]--;  
            events[end]++;  
  
            if (events[start] == 0) {  
                events.erase(start);  
            }  
  
            if (events[end] == 0) {  
                events.erase(end);  
            }  
  
            return false;  
        }  
    }  
  
    return true;  
}
```

Used in My Calendar II.

12. Tie Handling in Sweep Line

Tie handling is one of the biggest sources of bugs.

Suppose one meeting ends at time 10 and another starts at time 10.

For half-open intervals:

[0,10) and [10,20)

They do not overlap.

So at time 10, the end event should be processed before the start event.

Using events:

{end, -1}
{start, +1}

works because:

-1 comes before +1

when sorted as pairs.

For closed intervals:

[0,10] and [10,20]

They overlap at 10.

So start may need to be processed before end, depending on problem.

Always decide based on interval type.

13. Major Sweep Line Variations

Max overlap	start +1, end -1	Meeting Rooms II
Capacity check	active load <= capacity	Car Pooling
Population count	birth +1, death -1	Maximum Population
Dynamic booking	map events	My Calendar II/III
Closed interval groups	active intervals with closed endpoint	Min Groups
Sparse difference	map coordinate deltas	Describe Painting
Active heights	multiset of heights	Skyline
Covered length	coordinate compression / segment tree	Rectangle Area II
Offline queries	sorted starts/ends + binary search	Flowers in Bloom

14. Full Problem Walkthrough 1 — Meeting Rooms II, Sweep Line Version

14.1 Problem Summary

Given meeting intervals, return minimum number of rooms needed.

Example:

```
intervals = [[0,30], [5,10], [15,20]]
```

Output:

```
2
```

14.2 How to Identify Sweep Line

Need maximum number of active meetings at any time.

Convert intervals into events.

Meetings are usually half-open:

```
[start, end)
```

So if one meeting ends at 10 and another starts at 10, one room can be reused.

14.3 C++ Code

```
class Solution {
public:
    int minMeetingRooms(vector<vector<int>>& intervals) {
        vector<pair<int, int>> events;

        for (auto& interval : intervals) {
            events.push_back({interval[0], +1});
            events.push_back({interval[1], -1});
        }

        sort(events.begin(), events.end());

        int active = 0;
        int rooms = 0;

        for (auto& [time, delta] : events) {
            active += delta;
            rooms = max(rooms, active);
        }

        return rooms;
    }
};
```

14.4 Dry Run

Input:

```
[[0,30], [5,10], [15,20]]
```

Events:

(0,+1), (30,-1)
(5,+1), (10,-1)
(15,+1), (20,-1)

Sorted:

(0,+1), (5,+1), (10,-1), (15,+1), (20,-1), (30,-1)

0 +1	1	1
5 +1	2	2
10 -1	1	2
15 +1	2	2
20 -1	1	2
30 -1	0	2

14.5 Complexity

Time:

$O(n \log n)$

Space:

$O(n)$

15. Full Problem Walkthrough 2 — LeetCode 1094: Car Pooling

15.1 Problem Summary

Trips are:

[numPassengers, from, to]

Passengers get in at from and get out at to.

Return true if passenger count never exceeds capacity.

15.2 How to Identify Sweep Line

Passenger count changes at pickup/dropoff points.

Use events:

from: +passengers

to: -passengers

Trip interval is half-open:

[from, to)

15.3 C++ Code With Map

```
class Solution {
public:
    bool carPooling(vector<vector<int>>& trips, int capacity) {
        map<int, int> events;

        for (auto& trip : trips) {
            int passengers = trip[0];
            int from = trip[1];
            int to = trip[2];

            events[from] += passengers;
            events[to] -= passengers;
        }

        int active = 0;

        for (auto& [location, delta] : events) {
            active += delta;

            if (active > capacity) {
                return false;
            }
        }

        return true;
    }
};
```

15.4 Complexity

Time:

$O(n \log n)$

Space:

$O(n)$

15.5 Difference Array Alternative

If coordinates are small, use:

```
vector<int> diff(1001, 0);
```

That gives $O(n + \text{maxCoordinate})$.

16. Full Problem Walkthrough 3 — LeetCode 1854: Maximum Population Year

16.1 Problem Summary

Each log is:

[birth, death]

A person is alive from birth year to death year exclusive:

[birth, death)

Return earliest year with maximum population.

16.2 How to Identify Sweep Line

Population changes at birth and death.

Use:

birth: +1
death: -1

16.3 C++ Code

```
class Solution {
public:
    int maximumPopulation(vector<vector<int>>& logs) {
        map<int, int> events;

        for (auto& log : logs) {
            int birth = log[0];
            int death = log[1];

            events[birth]++;
            events[death]--;
        }

        int active = 0;
        int bestPopulation = 0;
        int bestYear = 0;

        for (auto& [year, delta] : events) {
            active += delta;

            if (active > bestPopulation) {
                bestPopulation = active;
            }
        }

        return bestYear;
    }
};
```

```
        bestYear = year;
    }
}

return bestYear;
}
};
```

16.4 Complexity

Time:

$O(n \log n)$

Space:

$O(n)$

16.5 Difference Array Alternative

Since years are usually limited from 1950 to 2050, difference array is simpler.

17. Full Problem Walkthrough 4 — LeetCode 729: My Calendar I

17.1 Problem Summary

Implement booking system.

Book interval:

[start, end)

Return false if it overlaps existing event.

17.2 Sweep / Ordered Interval Idea

This is dynamic interval overlap.

Store booked intervals sorted by start.

For a new interval, only previous and next intervals can overlap.

17.3 C++ Code

```
class MyCalendar {
private:
    map<int, int> intervals;

public:
    MyCalendar() {}

    bool book(int start, int end) {
        auto next = intervals.lower_bound(start);
```

```

    if (next != intervals.end() && next->first < end) {
        return false;
    }

    if (next != intervals.begin()) {
        auto prevIt = prev(next);

        if (prevIt->second > start) {
            return false;
        }
    }

    intervals[start] = end;
    return true;
}
};

```

17.4 Complexity

Each booking:

$O(\log n)$

Space:

$O(n)$

18. Full Problem Walkthrough 5 — LeetCode 731: My Calendar II

18.1 Problem Summary

Bookings can overlap at most twice.

Triple booking is not allowed.

Return false if new booking creates triple overlap.

18.2 How to Identify Sweep Line

Dynamic booking with active count.

Use ordered map events.

After adding event, scan active count.

If active reaches 3, undo.

18.3 C++ Code

```

class MyCalendarTwo {
private:
    map<int, int> events;

public:

```

```

MyCalendarTwo() {
}

bool book(int start, int end) {
    events[start]++;
    events[end]--;

    int active = 0;

    for (auto& [time, delta] : events) {
        active += delta;

        if (active >= 3) {
            events[start]--;
            events[end]++;

            if (events[start] == 0) {
                events.erase(start);
            }

            if (events[end] == 0) {
                events.erase(end);
            }

            return false;
        }
    }

    return true;
}
};

```

18.4 Complexity

Each booking:

$O(n)$

Space:

$O(n)$

18.5 Common Mistakes

- Forgetting to undo failed booking
- Not erasing zero-delta events
- Treating end as inclusive

19. Full Problem Walkthrough 6 — LeetCode 732: My Calendar III

19.1 Problem Summary

Return maximum number of overlapping events after each booking.

19.2 How to Identify Sweep Line

Same event map as My Calendar II.

But instead of rejecting, return max active count.

19.3 C++ Code

```
class MyCalendarThree {
private:
    map<int, int> events;

public:
    MyCalendarThree() {
    }

    int book(int start, int end) {
        events[start]++;
        events[end]--;

        int active = 0;
        int best = 0;

        for (auto& [time, delta] : events) {
            active += delta;
            best = max(best, active);
        }

        return best;
    }
};
```

19.4 Complexity

Each booking:

$O(n)$

Space:

$O(n)$

20. Full Problem Walkthrough 7 — LeetCode 2406: Divide Intervals Into Minimum Number of Groups

20.1 Problem Summary

Divide intervals into minimum groups such that no intervals in the same group overlap.

Intervals are closed.

Example:

[1,5] and [5,8]

They overlap.

20.2 How to Identify Sweep Line

Minimum groups equals maximum overlap.

For closed intervals, an interval ending at 5 is still active when another starts at 5.

20.3 C++ Code With Events

To handle closed intervals using integer coordinates:

```
class Solution {
public:
    int minGroups(vector<vector<int>>& intervals) {
        map<int, int> events;

        for (auto& interval : intervals) {
            int start = interval[0];
            int end = interval[1];

            events[start]++;
            events[end + 1]--;
        }

        int active = 0;
        int best = 0;

        for (auto& [point, delta] : events) {
            active += delta;
            best = max(best, active);
        }

        return best;
    }
};
```

20.4 Complexity

Time:

$O(n \log n)$

Space:

$O(n)$

20.5 Important Note

This end + 1 method works because interval coordinates are integers.

For non-integer coordinates, use event tie-breaking instead.

21. Full Problem Walkthrough 8 — LeetCode 2251: Number of Flowers in Full Bloom

21.1 Problem Summary

Each flower blooms during interval:

[start, end]

For each person arrival time, return how many flowers are blooming.

21.2 Sweep / Binary Search Idea

For time t :

blooming = flowers started by t - flowers ended before t

Since intervals are closed:

started = count(start $\leq t$)

ended = count(end $< t$)

Use sorted starts and ends.

21.3 C++ Code

```
class Solution {
public:
    vector<int> fullBloomFlowers(vector<vector<int>>& flowers, vector<int>&
people) {
        vector<int> starts;
        vector<int> ends;

        for (auto& flower : flowers) {
            starts.push_back(flower[0]);
            ends.push_back(flower[1]);
        }
    }
};
```

```

sort(starts.begin(), starts.end());
sort(ends.begin(), ends.end());

vector<int> answer;

for (int time : people) {
    int started = upper_bound(starts.begin(), starts.end(), time) - starts.begin();
    int ended = lower_bound(ends.begin(), ends.end(), time) - ends.begin();

    answer.push_back(started - ended);
}

return answer;
}
};

```

21.4 Complexity

Time:

$O((n + q) \log n)$

Space:

$O(n)$

22. Full Problem Walkthrough 9 — LeetCode 1943: Describe the Painting

22.1 Problem Summary

Each segment paints interval:

$[start, end)$

with color value.

If multiple colors overlap, their values add.

Return non-overlapping intervals with total color value.

22.2 How to Identify Sweep Line

Each segment adds color at start and removes color at end.

Use map events:

start: +color

end: -color

Between two consecutive event points, color sum is constant.

22.3 C++ Code

```

class Solution {
public:
    vector<vector<long long>> splitPainting(vector<vector<int>>& segments) {
        map<int, long long> events;

        for (auto& segment : segments) {
            int start = segment[0];
            int end = segment[1];
            int color = segment[2];

            events[start] += color;
            events[end] -= color;
        }

        vector<vector<long long>> result;

        long long activeColor = 0;
        int prev = -1;

        for (auto& [point, delta] : events) {
            if (prev != -1 && activeColor > 0 && prev < point) {
                result.push_back({prev, point, activeColor});
            }

            activeColor += delta;
            prev = point;
        }

        return result;
    }
};

```

22.4 Dry Run Concept

If active color is 7 from coordinate 3 to 5, output:

[3,5,7]

At next event, active color changes.

22.5 Complexity

Time:

$O(n \log n)$

Space:

$O(n)$

23. Full Problem Walkthrough 10 — Amount of New Area Painted Each Day

23.1 Problem Summary

Each day paints interval:

[start, end)

Return how much new area is painted each day.

Already painted area does not count again.

23.2 How to Identify Interval / Sweep Structure

Need maintain union of already painted intervals.

For each new interval, compute uncovered parts and merge.

Use ordered map of disjoint painted intervals.

23.3 C++ Code

```
vector<int> amountPainted(vector<vector<int>>& paint) {
    map<int, int> intervals;
    vector<int> result;

    for (auto& p : paint) {
        int start = p[0];
        int end = p[1];

        int added = 0;

        auto it = intervals.lower_bound(start);

        if (it != intervals.begin()) {
            auto prevIt = prev(it);

            if (prevIt->second >= start) {
                it = prevIt;
            }
        }

        int newStart = start;
        int newEnd = end;

        while (it != intervals.end() && it->first < end) {
            int s = it->first;
```

```

    int e = it->second;

    if (e < start) {
        ++it;
        continue;
    }

    if (s > newStart) {
        added += s - newStart;
    }

    newStart = max(newStart, e);
    newEnd = max(newEnd, e);

    auto eraselt = it;
    ++it;
    intervals.erase(eraselt);
}

if (newStart < end) {
    added += end - newStart;
}

intervals[start] = max(end, newEnd);

result.push_back(added);
}

return result;
}

```

23.4 Complexity

Each interval may be inserted and erased.

Amortized:

$O(n \log n)$

Space:

$O(n)$

23.5 Note

This problem is more of a dynamic interval-union problem than a basic sweep line, but it uses the same timeline idea.

24. Full Problem Walkthrough 11 — LeetCode 218: The Skyline Problem

24.1 Problem Summary

Each building is:

[left, right, height]

Return skyline key points.

24.2 How to Identify Sweep Line

Buildings start and end along x-axis.

The skyline height at any x is the maximum active building height.

Need sweep events and maintain active heights.

Use multiset.

24.3 Event Design

For each building:

left event: add height

right event: remove height

To sort correctly:

- start events should be processed before end events at same x
- taller start should be processed first
- shorter end should be processed first

A common encoding:

start event: -height

end event: +height

24.4 C++ Code

```
class Solution {
public:
    vector<vector<int>> getSkyline(vector<vector<int>>& buildings) {
        vector<pair<int, int>> events;

        for (auto& b : buildings) {
            int left = b[0];
            int right = b[1];
            int height = b[2];

            events.push_back({left, -height});
            events.push_back({right, height});
        }

        sort(events.begin(), events.end());
```

```

multiset<int> activeHeights;
activeHeights.insert(0);

int prevHeight = 0;

vector<vector<int>> result;

for (auto& [x, h] : events) {
    if (h < 0) {
        activeHeights.insert(-h);
    } else {
        auto it = activeHeights.find(h);

        if (it != activeHeights.end()) {
            activeHeights.erase(it);
        }
    }

    int currentHeight = *activeHeights.rbegin();

    if (currentHeight != prevHeight) {
        result.push_back({x, currentHeight});
        prevHeight = currentHeight;
    }
}

return result;
};

```

24.5 Why Multiset?

Multiple buildings can have the same height.

multiset allows duplicates and lets us remove one copy.

24.6 Complexity

Time:

$O(n \log n)$

Space:

$O(n)$

24.7 Common Mistakes

- Using set instead of multiset
- Removing all copies of a height
- Wrong event sorting

- Creating duplicate key points

25. Full Problem Walkthrough 12 — Rectangle Area II

25.1 Problem Summary

Given rectangles, return total area covered by union of rectangles.

Rectangles may overlap.

25.2 How to Identify Sweep Line

This is 2D sweep line.

Sweep over x-coordinates.

Between consecutive x events, active y-intervals determine covered height.

Area contribution:

```
coveredYLength * deltaX
```

25.3 High-Level Algorithm

For each rectangle [x1, y1, x2, y2]:

Create events:

```
x1: add y-interval [y1, y2]
```

```
x2: remove y-interval [y1, y2]
```

Process x in sorted order.

At each x:

26. Add area from previous x to current x using current active y coverage.
27. Apply all events at current x.
28. Continue.

25.4 Simple C++ Version

This version recomputes y-union each event. It is easier to understand, but not the most optimized.

```
class Solution {
public:
    static const int MOD = 1000000007;

    long long getYCoverage(vector<pair<int, int>>& intervals) {
        if (intervals.empty()) {
            return 0;
        }

        sort(intervals.begin(), intervals.end());

        long long total = 0;
        int start = intervals[0].first;
        int end = intervals[0].second;
```

```

for (int i = 1; i < intervals.size(); i++) {
    if (intervals[i].first > end) {
        total += end - start;
        start = intervals[i].first;
        end = intervals[i].second;
    } else {
        end = max(end, intervals[i].second);
    }
}

total += end - start;

return total;
}

int rectangleArea(vector<vector<int>>& rectangles) {
    vector<vector<int>> events;

    for (auto& r : rectangles) {
        events.push_back({r[0], 1, r[1], r[3]});
        events.push_back({r[2], -1, r[1], r[3]});
    }

    sort(events.begin(), events.end());

    vector<pair<int, int>> active;
    long long area = 0;
    int prevX = events[0][0];

    int i = 0;

    while (i < events.size()) {
        int x = events[i][0];

        long long width = x - prevX;
        long long height = getYCoverage(active);

        area = (area + width * height) % MOD;

        while (i < events.size() && events[i][0] == x) {
            int type = events[i][1];

```

```

int y1 = events[i][2];
int y2 = events[i][3];

if (type == 1) {
    active.push_back({y1, y2});
} else {
    for (int j = 0; j < active.size(); j++) {
        if (active[j].first == y1 && active[j].second == y2) {
            active.erase(active.begin() + j);
            break;
        }
    }
}

i++;
}

prevX = x;
}

return area;
}
};

```

25.5 Complexity

This simple version can be:

$O(n^2 \log n)$

Optimized version uses coordinate compression + segment tree.

That belongs later in Segment Tree / Sweep Line advanced coverage.

26. Full Problem Walkthrough 13 — Corporate Flight Bookings as Sweep / Difference

26.1 Problem Summary

Each booking:

[first, last, seats]

adds seats to all flights from first to last inclusive.

26.2 Sweep / Difference Idea

Use start and stop events:

```
first: +seats  
last + 1: -seats
```

Then prefix scan.

26.3 C++ Code

```
class Solution {  
public:  
    vector<int> corpFlightBookings(vector<vector<int>>& bookings, int n) {  
        vector<int> diff(n + 1, 0);  
  
        for (auto& booking : bookings) {  
            int first = booking[0] - 1;  
            int last = booking[1] - 1;  
            int seats = booking[2];  
  
            diff[first] += seats;  
            diff[last + 1] -= seats;  
        }  
  
        vector<int> answer(n);  
        int active = 0;  
  
        for (int i = 0; i < n; i++) {  
            active += diff[i];  
            answer[i] = active;  
        }  
  
        return answer;  
    }  
};
```

26.4 Key Lesson

Difference array is sweep line on dense integer coordinates.

27. Full Problem Walkthrough 14 — Range Addition

27.1 Problem Summary

Initial array of length n is all zero.

Each update:

```
[l, r, val]
```

adds val to every index from l to r.

27.2 Sweep / Difference Idea

For inclusive [l, r]:

l: +val
r + 1: -val

27.3 C++ Code

```
vector<int> getModifiedArray(int length, vector<vector<int>>& updates) {  
    vector<int> diff(length + 1, 0);  
  
    for (auto& update : updates) {  
        int l = update[0];  
        int r = update[1];  
        int val = update[2];  
  
        diff[l] += val;  
        diff[r + 1] -= val;  
    }  
  
    vector<int> result(length);  
    int active = 0;  
  
    for (int i = 0; i < length; i++) {  
        active += diff[i];  
        result[i] = active;  
    }  
  
    return result;  
}
```

28. Full Problem Walkthrough 15 — LeetCode 2021: Brightest Position on Street

28.1 Problem Summary

Each light is represented as:

[position, range]

It illuminates:

[position - range, position + range]

Return the smallest position with maximum brightness.

28.2 How to Identify Sweep Line

Each light contributes +1 brightness over an interval.

Need max active count.

Intervals are integer and closed.

Use events:

left: +1

right + 1: -1

28.3 C++ Code

```
int brightestPosition(vector<vector<int>>& lights) {
    map<int, int> events;

    for (auto& light : lights) {
        int position = light[0];
        int range = light[1];

        int left = position - range;
        int right = position + range;

        events[left]++;
        events[right + 1]--;
    }

    int active = 0;
    int best = 0;
    int answer = 0;

    for (auto& [position, delta] : events) {
        active += delta;

        if (active > best) {
            best = active;
            answer = position;
        }
    }

    return answer;
}
```

28.4 Complexity

Time:

$O(n \log n)$

Space:

$O(n)$

29. Common Sweep Line Edge Cases

29.1 Empty Events

If no intervals exist, return 0 or empty depending on problem.

29.2 Same Start and End

For half-open intervals:

`[start, start)`

has zero length.

Usually ignore or handle naturally.

29.3 Multiple Events at Same Coordinate

Using map automatically combines deltas.

Using vector events requires careful sorting/tie handling.

29.4 Closed vs Half-Open

Closed intervals may require:

`end + 1`

if integer coordinates.

Half-open intervals use:

`end`

directly.

29.5 Large Coordinates

Use map, not array.

29.6 Large Active Sum

Use long long if active cost/color/area can become large.

30. Common Mistakes and Wrong Code

Mistake 1 — Wrong End Event

For half-open interval `[start, end)`:

`events[end]--;`

For closed integer interval `[start, end]`:

`events[end + 1]--;`

Mistake 2 — Wrong Tie Handling

For meetings, end at time t should happen before start at time t .

Otherwise you overcount rooms.

Mistake 3 — Forgetting to Undo Calendar Booking

In My Calendar II, failed booking must be removed.

Mistake 4 — Using Difference Array for Huge Coordinates

If coordinate can be $1e9$, do not allocate array.

Use map.

Mistake 5 — Using Set Instead of Multiset in Skyline

Duplicate heights matter.

Use multiset.

Mistake 6 — Not Grouping Same-X Events in Geometry

For skyline and rectangle problems, multiple events at same coordinate can affect output.

31. Interview Explanation Templates

31.1 Basic Sweep Line

I convert each interval into a start event and an end event. Then I sort events by coordinate and scan from left to right while maintaining the number of active intervals.

31.2 Meeting Rooms

Each meeting adds one active room at its start and removes one at its end. The maximum active count during the sweep is the number of rooms needed.

31.3 Car Pooling

Each trip adds passengers at pickup and removes passengers at dropoff. While sweeping locations, if active passengers exceed capacity, return false.

31.4 My Calendar II

I temporarily add the new booking as events. Then I sweep active bookings. If active reaches 3, the booking creates a triple overlap, so I undo the events and return false.

31.5 Skyline

I sweep building start/end events while maintaining active building heights in a multiset. Whenever the maximum active height changes, that x-coordinate becomes a skyline key point.

32. Complexity Patterns

Basic max overlap	$O(n \log n)$	$O(n)$
Map sweep line	$O(n \log n)$	$O(n)$
Difference array sweep	$O(n + C)$	$O(C)$
Meeting Rooms II	$O(n \log n)$	$O(n)$

Problem Type	Time	Space
Car Pooling map	$O(n \log n)$	$O(n)$
My Calendar I	$O(\log n)$ per booking	$O(n)$
My Calendar II	$O(n)$ per booking	$O(n)$
My Calendar III	$O(n)$ per booking	$O(n)$
Skyline	$O(n \log n)$	$O(n)$
Rectangle Area II simple	$O(n^2 \log n)$	$O(n)$
Flowers in Bloom	$O((n + q) \log n)$	$O(n)$

33. Time-Boxed Practice Guidance

Intro Sweep Line

Target:

20–30 minutes

Problems:

- Meeting Rooms II, sweep version
- Car Pooling
- Maximum Population Year
- Brightest Position

Medium Sweep Line

Target:

30–50 minutes

Problems:

- My Calendar II
- My Calendar III
- Divide Intervals Into Groups
- Number of Flowers in Full Bloom
- Describe the Painting

Hard Sweep Line

Target:

60–90 minutes

Problems:

- The Skyline Problem
- Rectangle Area II
- Amount of New Area Painted Each Day
- Count Integers in Intervals
- Falling Squares

34. Expanded Practice Set

34.1 Must-Do Medium

- 29. Meeting Rooms II, sweep line
- 30. Car Pooling
- 31. Maximum Population Year
- 32. Brightest Position on Street
- 33. Divide Intervals Into Minimum Number of Groups
- 34. My Calendar I
- 35. My Calendar II
- 36. My Calendar III
- 37. Number of Flowers in Full Bloom
- 38. Describe the Painting

34.2 Strong Interview Practice

- 39. The Skyline Problem
- 40. Amount of New Area Painted Each Day
- 41. Employee Free Time
- 42. Range Module
- 43. Count Integers in Intervals
- 44. Corporate Flight Bookings
- 45. Range Addition
- 46. Merge Intervals + Sweep Comparison
- 47. Falling Squares

34.3 Hard / Stretch

- 48. Rectangle Area II
 - 49. The Skyline Problem
 - 50. Count Integers in Intervals
 - 51. Falling Squares
 - 52. Perfect Rectangle
 - 53. Meeting Rooms III
 - 54. My Calendar III
 - 55. Amount of New Area Painted Each Day
-

35. Suggested Study Order

Step 1 — Basic Event Counting

Start with:

- 56. Meeting Rooms II
- 57. Car Pooling
- 58. Maximum Population Year

Goal:

Understand start/end events and active count.

Step 2 — Sparse Event Maps

Solve:

- 59. My Calendar II
- 60. My Calendar III
- 61. Brightest Position

Goal:

Learn ordered map event storage.

Step 3 — Boundary and Closed Intervals

Solve:

- 62. Divide Intervals Into Groups
- 63. Number of Flowers in Full Bloom

Goal:

Master closed vs half-open boundaries.

Step 4 — Segment Output

Solve:

- 64. Describe the Painting
- 65. Employee Free Time

Goal:

Output intervals between events.

Step 5 — Advanced Geometry

Solve:

- 66. The Skyline Problem
- 67. Rectangle Area II

Goal:

Maintain active sets and covered length.

36. Pattern Decision Table

maximum overlap	events + active count	Meeting Rooms II
capacity over route	pickup/dropoff events	Car Pooling
population over years	birth/death events	Maximum Population
dynamic double booking	map events + active check	My Calendar II
maximum booking count	map events + max active	My Calendar III
closed interval groups	end + 1 event or tie handling	Min Groups
count active at query time	sorted starts/ends	Flowers in Bloom
colored segment sums	map deltas and output between events	Describe Painting
skyline	events + multiset active heights	Skyline
rectangle union area	sweep x + active y coverage	Rectangle Area II

Problem Clue	Sweep Strategy	Example
dense integer updates	difference array	Range Addition

37. Mini Problem 1 — Maximum Overlap Count

```
int maximumOverlap(vector<vector<int>>& intervals) {
    vector<pair<int, int>> events;

    for (auto& interval : intervals) {
        events.push_back({interval[0], +1});
        events.push_back({interval[1], -1});
    }

    sort(events.begin(), events.end());

    int active = 0;
    int best = 0;

    for (auto& [time, delta] : events) {
        active += delta;
        best = max(best, active);
    }

    return best;
}
```

38. Mini Problem 2 — Earliest Time With Maximum Active Count

```
int earliestMaxOverlapTime(vector<vector<int>>& intervals) {
    map<int, int> events;

    for (auto& interval : intervals) {
        events[interval[0]]++;
        events[interval[1]]--;
    }
}
```

```

int active = 0;
int best = 0;
int answer = 0;

for (auto& [time, delta] : events) {
    active += delta;

    if (active > best) {
        best = active;
        answer = time;
    }
}

return answer;
}

```

39. Mini Problem 3 — Output Active Segments

Given events, output ranges where active count is positive.

```

vector<vector<int>> activeSegments(map<int, int>& events) {
    vector<vector<int>> result;

    int active = 0;
    int prev = -1;

    for (auto& [point, delta] : events) {
        if (prev != -1 && active > 0 && prev < point) {
            result.push_back({prev, point});
        }

        active += delta;
        prev = point;
    }

    return result;
}

```

Used in:

- Describe the Painting
- union of intervals
- active timeline output

40. Mini Problem 4 — Difference Array as Sweep

```
vector<int> applyUpdates(int n, vector<vector<int>>& updates) {  
    vector<int> diff(n + 1, 0);  
  
    for (auto& update : updates) {  
        int l = update[0];  
        int r = update[1];  
        int val = update[2];  
  
        diff[l] += val;  
        diff[r + 1] -= val;  
    }  
  
    vector<int> result(n);  
    int active = 0;  
  
    for (int i = 0; i < n; i++) {  
        active += diff[i];  
        result[i] = active;  
    }  
  
    return result;  
}
```

41. Common Edge Case Checklist

Before submitting sweep line code, test:

- no intervals
- one interval
- same start times
- same end times
- start equals previous end
- closed interval touching
- half-open interval touching
- nested intervals
- duplicate intervals
- large coordinates
- negative coordinates

events cancel to zero
multiple events at same coordinate

42. What to Master Before Chapter 34

Before moving to Dynamic Programming Foundations, make sure you can:

- Convert intervals to events
- Use start +1 and end -1
- Distinguish closed vs half-open intervals
- Handle event tie-breaking
- Use map for sparse events
- Use vector events for offline sweep
- Compute maximum active count
- Check capacity with active count
- Solve Meeting Rooms II using sweep
- Solve Car Pooling
- Solve My Calendar II
- Solve My Calendar III
- Solve Number of Flowers in Bloom
- Understand Skyline event processing
- Understand difference array as dense sweep line

43. Final Cheat Sheet

Half-Open Interval [start, end)

```
events[start]++;  
events[end]--;
```

Closed Integer Interval [start, end]

```
events[start]++;  
events[end + 1]--;
```

Basic Active Count

```
int active = 0;  
  
for (auto& [point, delta] : events) {  
    active += delta;  
    best = max(best, active);  
}
```

Vector Events

```
events.push_back({start, +1});
events.push_back({end, -1});
sort(events.begin(), events.end());
```

Map Events

```
map<int, int> events;
events[start]++;
events[end]--;
```

Skyline Events

```
events.push_back({left, -height});
events.push_back({right, height});
```

Calendar Undo

```
events[start]--;
events[end]++;

if (events[start] == 0) {
    events.erase(start);
}

if (events[end] == 0) {
    events.erase(end);
}
```

44. Core Takeaway

Sweep Line is event-based interval processing.
The core transformation is:

interval -> start event + end event

Then scan from left to right while maintaining active state.
Use sweep line when the problem asks:

How many intervals are active here?
When does active count exceed a limit?
Where does the active value change?
What is the maximum overlap?
What segments are covered?

The hardest part is usually not the scan.
The hardest part is choosing the correct event representation and tie-breaking rule.
Always ask:

Is the interval closed or half-open?
Should start happen before end at the same coordinate?

Should end happen before start?
Are coordinates dense or sparse?

Once those are clear, sweep line becomes a powerful and reusable pattern.